

# SHARPER RETRIEVAL CHEATSHEET

AI Engineer Ready · Week 6 — RAG Optimization + Evaluation + LangSmith

Golden Rule -> ask it more ways, then read only what matters -- widen recall, then strip the noise

## 1. Why Naive Retrieval Breaks

A basic RAG retriever takes the user's exact question, embeds it, and does one similarity search. That works in a demo and quietly fails in real use, for three reasons:

| Problem                | What It Looks Like                                                                                    | Why It Hurts                                                                                                                         |
|------------------------|-------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| One phrasing, one shot | Plain similarity search only searches the words you typed, not the words the document actually uses.  | A single odd phrasing can miss a chunk that clearly contains the answer.                                                             |
| Noisy top-K            | The top matches often include half-relevant chunks that just add clutter for the LLM to wade through. | The LLM has to read through noise, which increases the chance it grabs the wrong sentence or gets distracted ("lost in the middle"). |
| No feedback loop       | Without a score, you can't tell if yesterday's tweak made retrieval better or quietly made it worse.  | Teams ship regressions because nobody was measuring retrieval quality in the first place.                                            |

A familiar example: a customer asks "I paid late, what happens?" but the policy document says "grace period". Same meaning, completely different words. One retriever call, one phrasing -- miss the wording, miss the answer.

## 2. Fix #1 -- Multi-Query Retriever

Instead of trusting the user's exact wording, ask an LLM to rewrite the question several different ways, search with every rewrite, then merge and de-duplicate the results. Simply: ask it more ways.

```
$ 1 question
$ -> LLM writes 3-4 phrasings
$ -> search each phrasing separately
$ -> merge + de-dupe the retrieved chunks
```

```
$ "What happens if I pay late?"
$ -> "grace period"
$ -> "late premium penalty"
$ -> "missed due date consequences"
```

```
$ from langchain.retrievers.multi_query import MultiQueryRetriever

$ mq_retriever = MultiQueryRetriever.from_llm(
$ retriever=base_retriever,
$ llm=llm,
$ )
$ docs = mq_retriever.invoke("I paid my premium late, what happens?")
```

## 3. Fix #2 -- Contextual Compression

Retrieve as usual, then trim every chunk down to only the sentences that actually answer the question. A 400-word policy clause becomes the one sentence that matters. Simply: read only what matters.

# SHARPER RETRIEVAL CHEATSHEET

AI Engineer Ready · Week 6 — RAG Optimization + Evaluation + LangSmith

Golden Rule -> ask it more ways, then read only what matters -- widen recall, then strip the noise

```
$ Top-K raw chunks (long)
$ -> LLM reads each chunk + the question
$ -> keeps only the relevant lines
$ -> compressed context (short) goes to the final prompt
```

```
$ from langchain.retrievers import ContextualCompressionRetriever
$ from langchain.retrievers.document_compressors import LLMChainExtractor

$ compressor = LLMChainExtractor.from_llm(llm)
$ compression_retriever = ContextualCompressionRetriever(
$ base_compressor=compressor,
$ base_retriever=base_retriever,
$ )
```

## 4. Multi-Query vs. Contextual Compression

|            | Multi-Query Retriever                         | Contextual Compression                                              |
|------------|-----------------------------------------------|---------------------------------------------------------------------|
| Fixes      | What you search for (recall)                  | What survives after you've found it (precision)                     |
| Extra cost | N extra LLM calls + N extra retrieval calls   | 1 extra LLM call per retrieved chunk                                |
| Best when  | Users phrase questions in many different ways | Chunks are long and mostly irrelevant text around one good sentence |

Both fixes slot into your existing retriever -- no rebuild required. Most real systems chain them together: multi-query widens the net, compression cleans what gets caught in it.

### Pitfall

Multi-Query means N extra LLM calls and N extra vector-store searches per user question -- watch latency and API cost, especially at scale. Contextual Compression can over-trim: if the compressor LLM is too aggressive, it can strip a sentence you actually needed.

### Remember

Multi-Query fixes what you search for. Contextual Compression fixes what survives after you've found it. Naive RAG isn't broken -- it's brittle.

# SHARPER RETRIEVAL CHEATSHEET

AI Engineer Ready · Week 6 — RAG Optimization + Evaluation + LangSmith

Golden Rule -> ask it more ways, then read only what matters -- widen recall, then strip the noise

## INTERVIEW PREP

### **Q: Why not just retrieve fewer chunks instead of compressing?**

A: Fewer chunks risks missing the answer entirely. Compression keeps the recall (you still retrieved broadly) but removes the noise after the fact.

### **Q: How does Multi-Query Retriever actually generate its extra queries?**

A: It sends your original question to an LLM with a prompt asking for several alternative phrasings, then runs the base retriever once per phrasing and merges the unique results.

### **Q: What's the main cost trade-off with Multi-Query?**

A: More recall and better coverage of wording variation, at the cost of extra LLM calls and extra retrieval latency for every single user question.

### **Q: If your RAG bot misses an answer you know is in the document, what's your first move?**

A: Check whether it's a wording mismatch (fix with Multi-Query) or a noisy top-K problem where the right chunk was retrieved but buried (fix with Contextual Compression or reranking).